

Playbook: Migrating Palantir Business Logic

The greatest migration risk isn't moving the data; it's translating the proprietary business logic locked inside Palantir's "walled garden." If the logic does not operate with 1-to-1 parity in GCP, trust in the migration collapses immediately.

This playbook establishes the strict mapping required to extract logic from Palantir's visual and serverless components and re-platform it securely and durably on Google Cloud.

1. Contour & Pipeline Builder (Visual ETL Logic)

Palantir allows analysts to build complex data transformations using "no-code/low-code" clicks. These clicks (Filter, Join, Group By) are stored internally as a JSON Directed Acyclic Graph (DAG).

Target GCP Service: BigQuery Standard SQL via Dataform (.sqlx)

Migration Strategy:

- **Extraction:** Use Palantir's APIs to extract the underlying JSON DAG representation of the Contour path.
- **Translation:** Pass the JSON through a compiler (`contour_to_sql_translator.py`) to generate raw BigQuery SQL.
- **Parity First:** We do *not* attempt to optimize the logic into BigQuery ML or Materialized Views initially. The goal is strict 1-to-1 fidelity so analysts see the exact outputs they expect. Optimization happens post-migration.

2. Code Repositories (PySpark & SQL)

Palantir Code Repositories run Apache Spark jobs under the hood. While PySpark is portable, Palantir's proprietary `Transform` decorators and ontology injections are not.

Target GCP Service: Dataproc Serverless (PySpark) / BigSpark

Migration Strategy:

- **Strip the Decorators:** Remove proprietary `@transform` and `@transform_df` wrappers.
- **Semantic Gating:** Ensure the PySpark logic produces the exact same dataset hash as it did in Foundry (using `pyspark_hash_validator.py`).
- **Execution:** Deploy as Dataproc Serverless batches orchestrated by Cloud Composer (Airflow) or Dataform.

3. Workshop Actions & Typescript Functions

Workshop Actions execute stateful, UI-driven mutations (e.g., "Approve Loan", "Update Shipping Status"). These are often complex Typescript functions that run statelessly but modify the ontology.

Target GCP Service: Cloud Run (FastAPI/Go/Node.js) + API Gateway

Migration Strategy:

- **Why Cloud Run?** While Cloud Functions are simpler, Workshop Actions are enterprise-grade workloads. Cloud Run provides better scaling, durability, containerization, and VPC integration.
- **CQRS Architecture:** As defined in the Execution Toolkit, Cloud Run acts as the Command layer, writing directly to Cloud SQL/Spanner (the ontology write-store), while BigQuery handles reads.
- **Future Extensibility:** By deploying logic as containerized REST APIs behind API Gateway, the client instantly unlocks the ability to share these actions with M&A targets or B2B partners—something Palantir heavily restricts.

4. Slate & Workshop UIs

Palantir provides drag-and-drop UI builders that bind directly to their ontology and action APIs.

Target GCP Service: React (Custom) or Streamlit (Fast Prototyping)

Migration Strategy:

- **Decoupling:** Because we migrated the underlying data to BigQuery and the mutation logic to Cloud Run, the UI layer is completely decoupled.
- **Implementation:** For executive dashboards and simple operational tools, Streamlit is the perfect rapid replacement. For complex, customer-facing applications, custom React frontends backed by the Cloud Run APIs are standard.